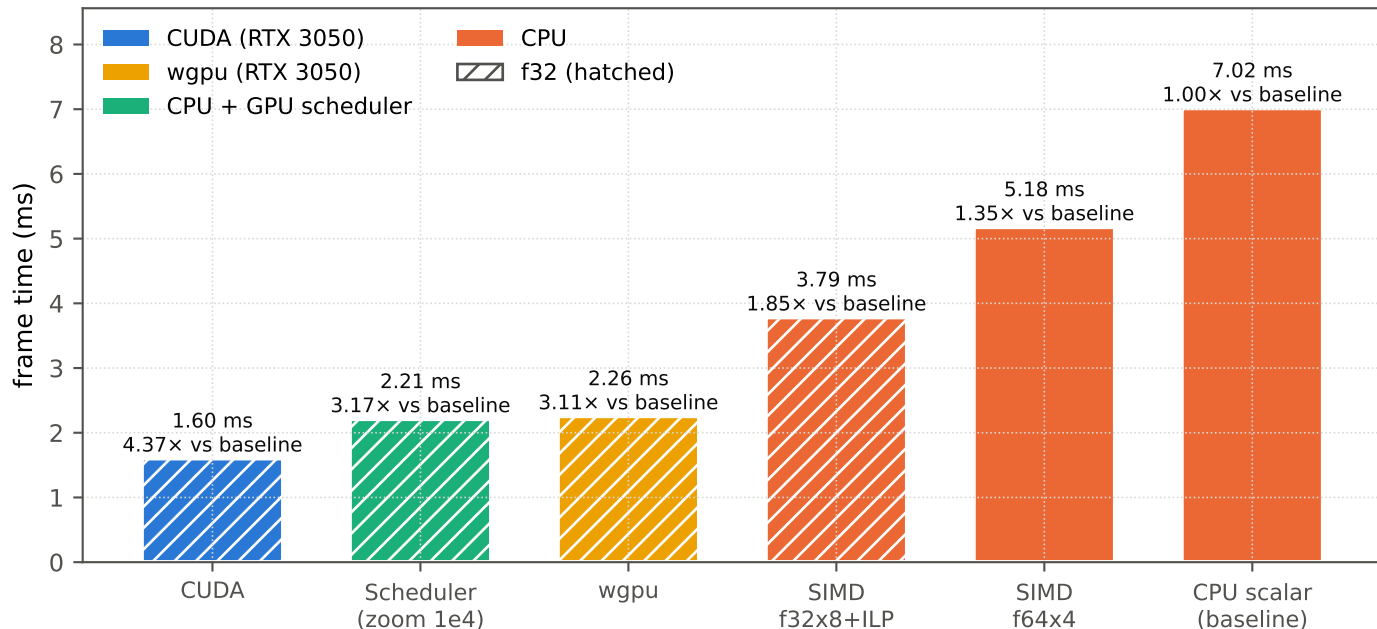


The adaptive scheduler wins exactly where the GPU loses its f32 fast path



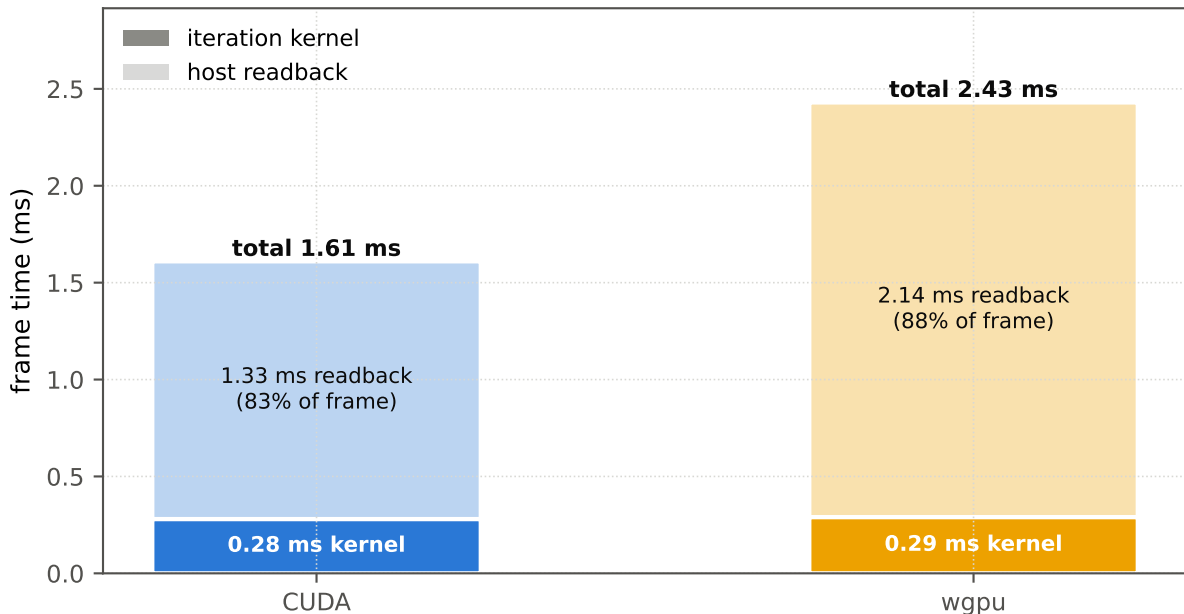
1920x1080 Mandelbrot, max_iter=1000. All three arms measured back-to-back in one criterion process, so session-to-session thermal drift cancels. Error bars are criterion's 95% confidence intervals. Wins past $1e7$ are real but marginal (1.02-1.16x): the load balancer still leaves ~45% unclaimed there.

Backend comparison — 1920×1080 Mandelbrot, max_iter=1000



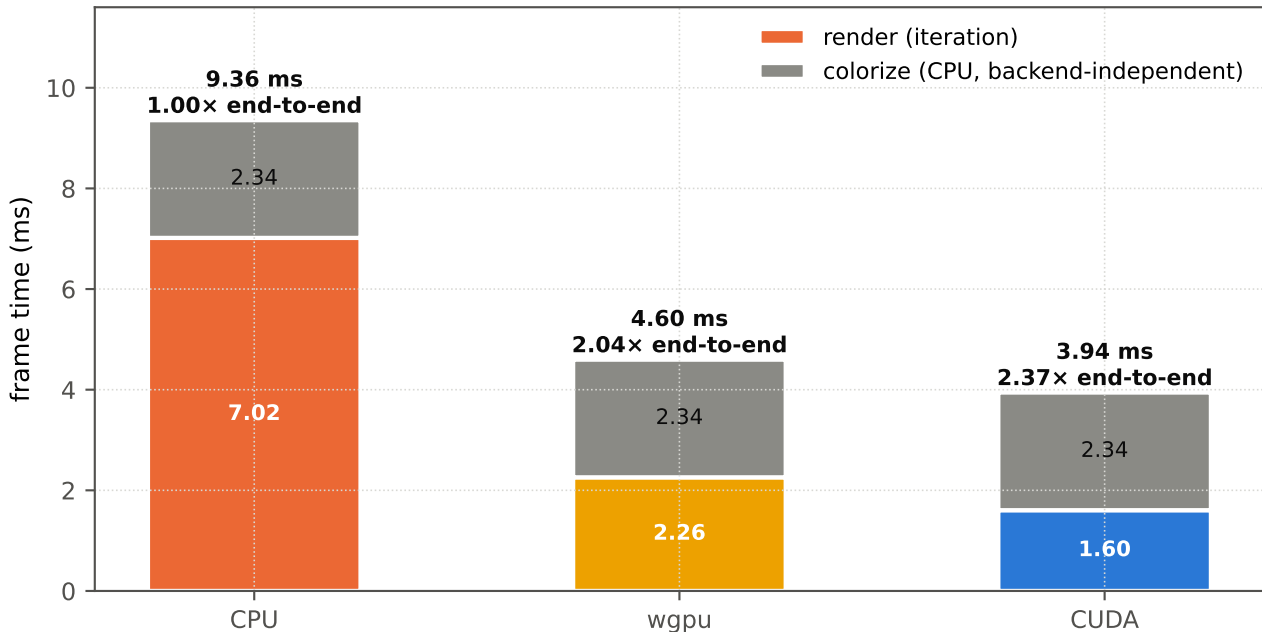
Hatched bars are f32, solid are f64 — not a like-for-like comparison across the two. The precision-controlled pairing is CUDA (1.61 ms) against the CPU's own best f32 path (3.79 ms) = 2.36×; against the f64 baseline it reads 4.37×.

A GPU frame is mostly data movement, not computation



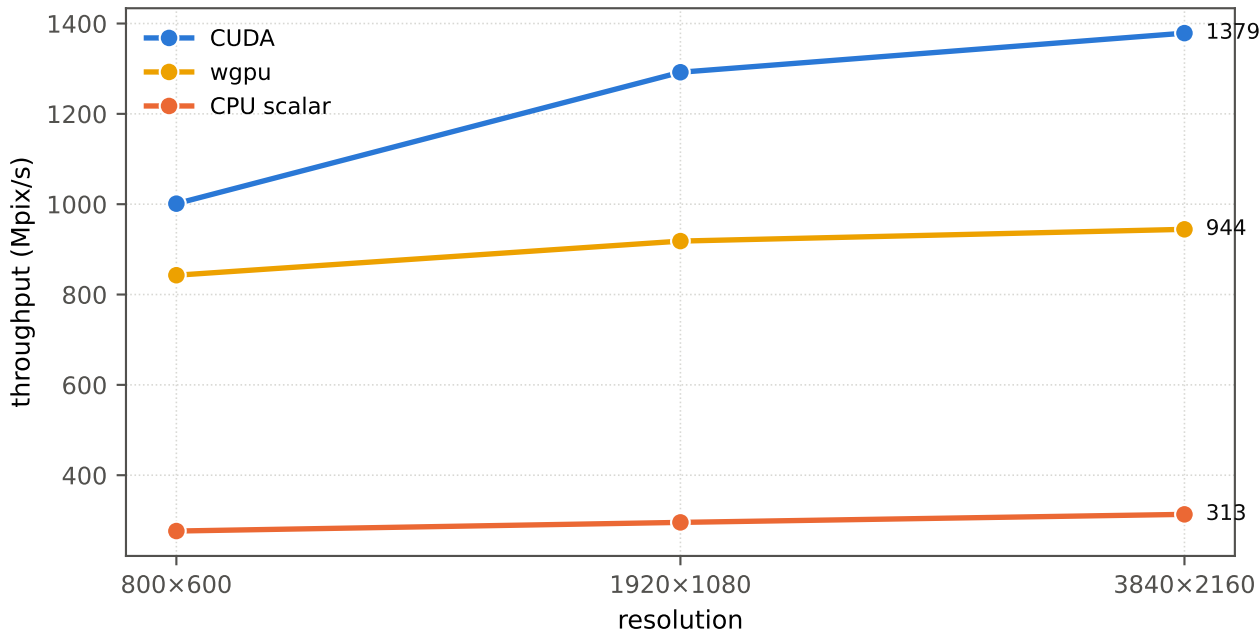
1920×1080 Mandelbrot, from `examples/gpu_probe.rs` (median of 5 process launches). The link is PCIe 3.0 ×8 — 7.9 GB/s theoretical, 6.7 measured page-locked, so the copy can only be made smaller, never faster. Halving the kernel (the fp64 bulb fix) raised the readback's share from 69% to 82%.

The CPU colour stage caps every backend's end-to-end gain



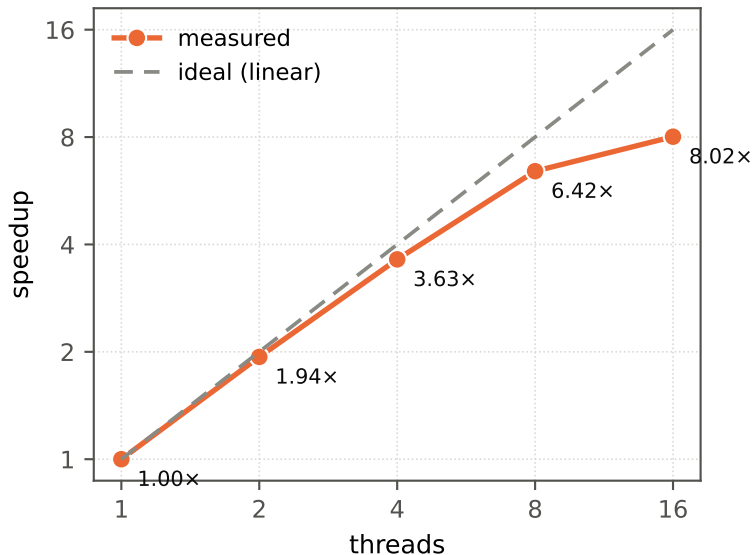
colorize() — histogram equalisation + palette LUT — is CPU-only and identical for every backend. It is 47% of the CUDA pipeline. Of a 4.97 ms CUDA pipeline roughly 0.3 ms is actual fractal iteration; moving this stage onto the GPU is worth 2.18x, more than any remaining kernel optimisation.

Throughput is flat in resolution — the kernel is embarrassingly parallel

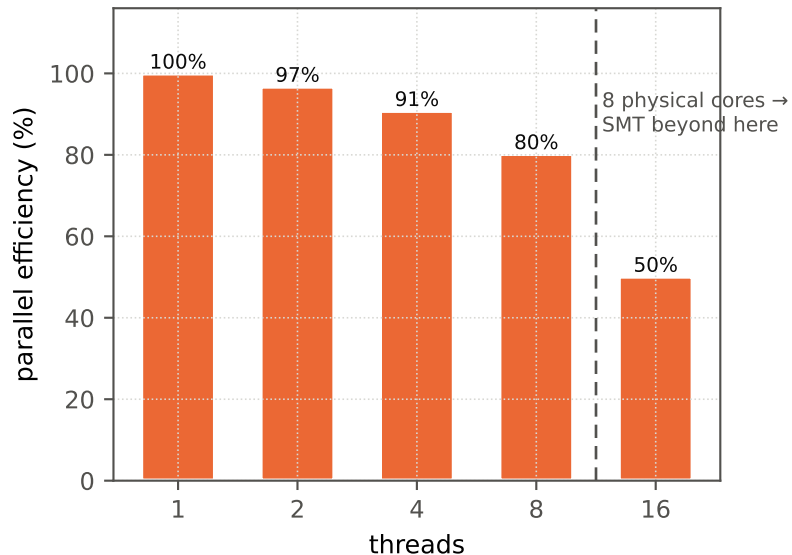


Mandelbrot, max_iter=1000. A per-pixel kernel with no shared state should hold constant Mpix/s as the frame grows; it does. The GPU curves rise slightly because their fixed per-frame costs — dispatch and the readback — amortise over more pixels.

Parallel speedup

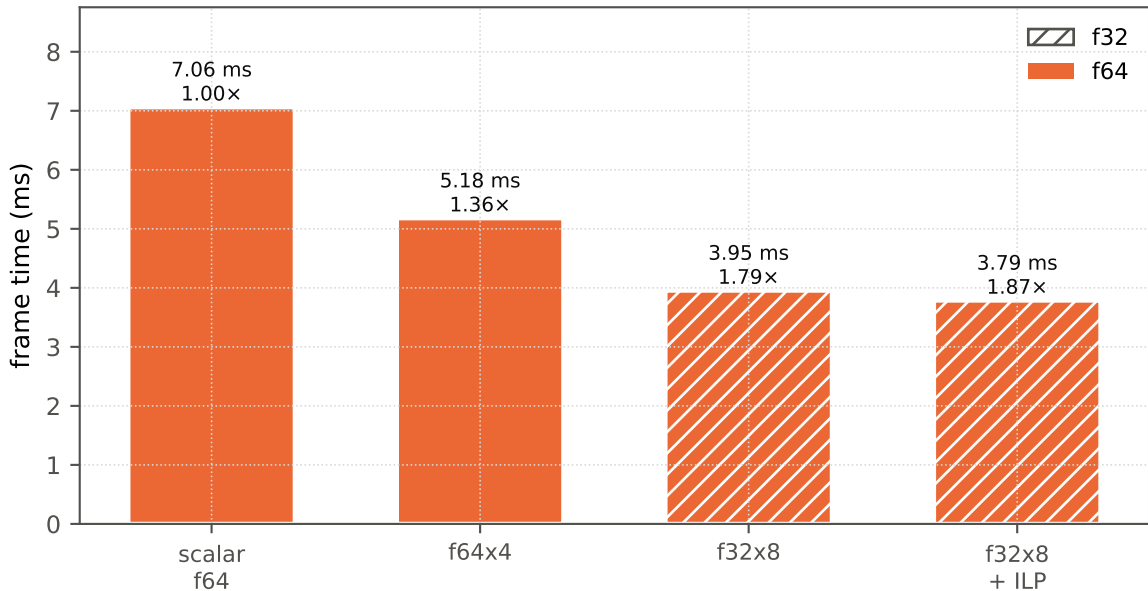


Efficiency



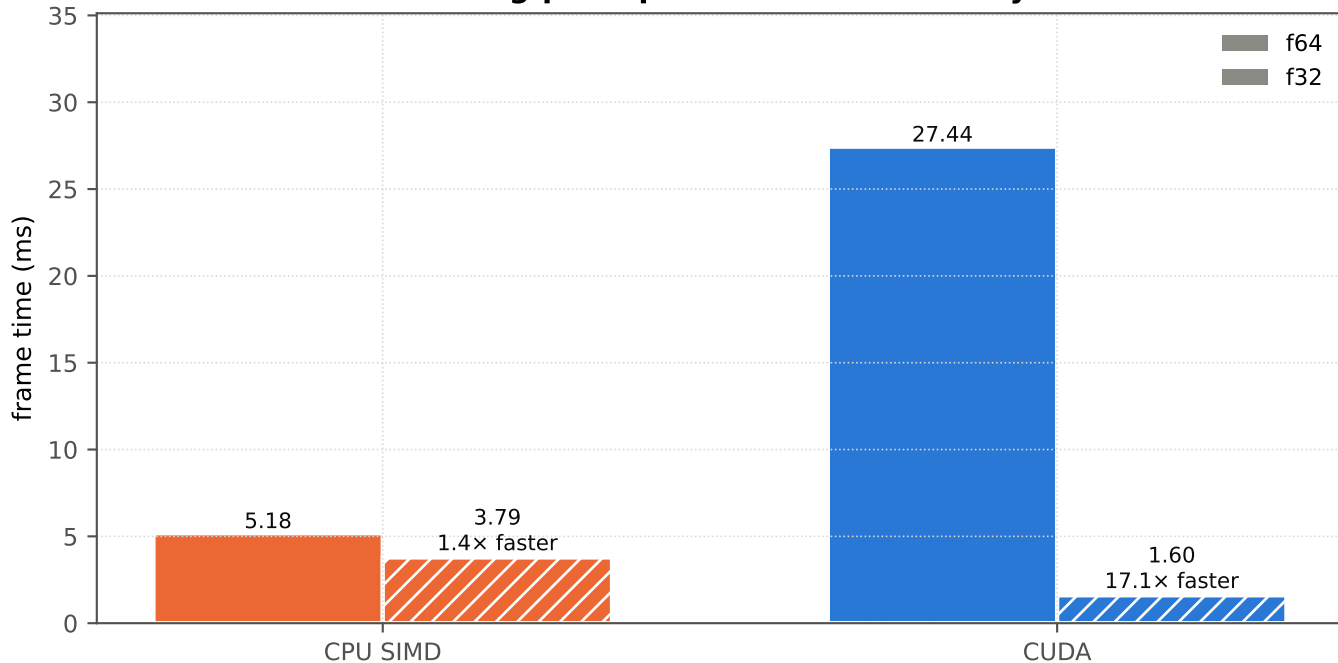
Mandelbrot 1920×1080, max_iter=1000, rayon. Efficiency holds above 90% to 4 threads and 80% at 8 — the Ryzen 7 5800H's physical core count — then halves at 16. That is the SMT signature: sibling threads share execution units, so the last doubling buys 25% rather than another 2x.

CPU vectorisation — AVX2, 1920×1080



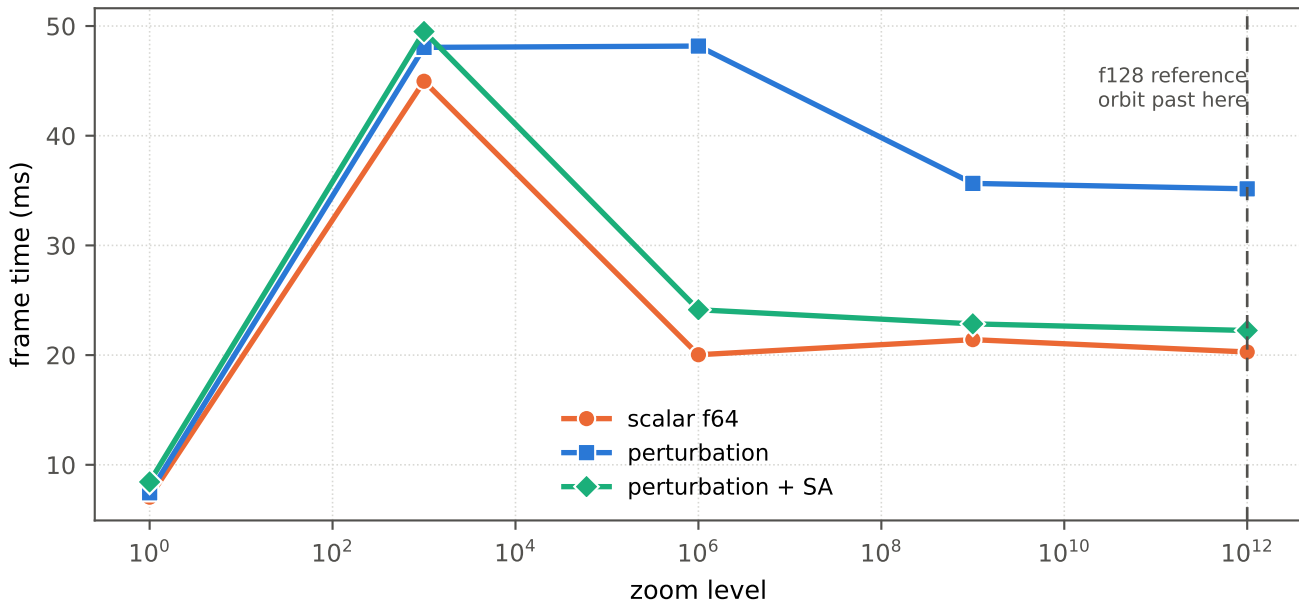
Vectorisation buys 1.87x over scalar. ILP — two interleaved f32x8 dependency chains, bit-identical output — adds only 4%, at the low end of expectations, because Mandelbrot's cardioid and bulb rejection already prune many pixels before the hot loop, leaving less latency to hide.

Floating-point precision: what f32 buys



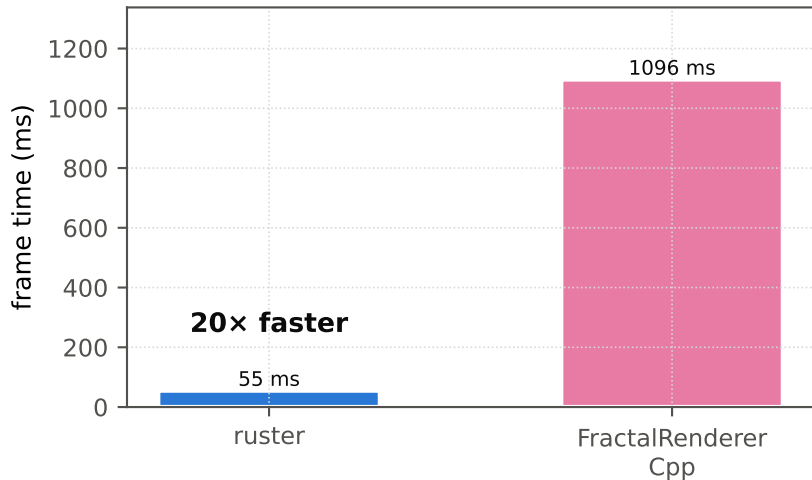
f32 is only NUMERICALLY VALID below zoom 1e6 — above it the mantissa can no longer separate adjacent pixels and the result is fast but wrong. CUDA's f64 penalty is severe because Ampere GeForce runs fp64 at 1/64 the fp32 rate; on the CPU the gap is far smaller. This asymmetry is the whole reason the scheduler has a regime where it wins.

Perturbation theory is a correctness feature, not a throughput one

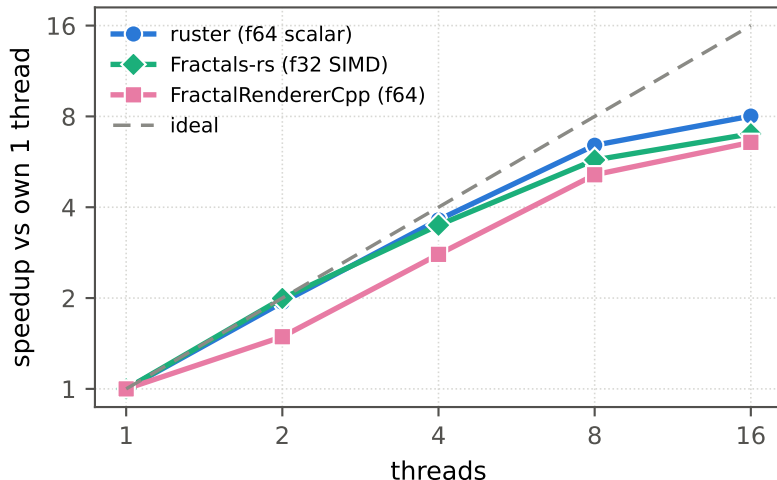


1920×1080, max_iter=1000, seahorse valley. Plain scalar f64 is faster than perturbation at every zoom tested — series approximation skips only 2-36 iterations of 1000 for this reference point. Perturbation earns its place past f64's $\sim 1e15$ ceiling, where scalar iteration cannot resolve individual pixels at all; that regime is beyond what this sweep measures.

Single-thread f64 scalar (fully controlled comparison)

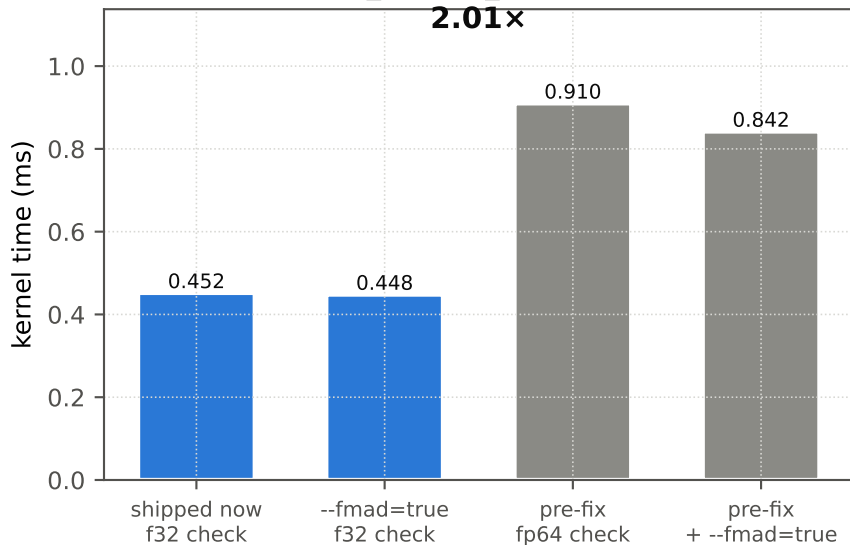


Parallel scaling (normalised)

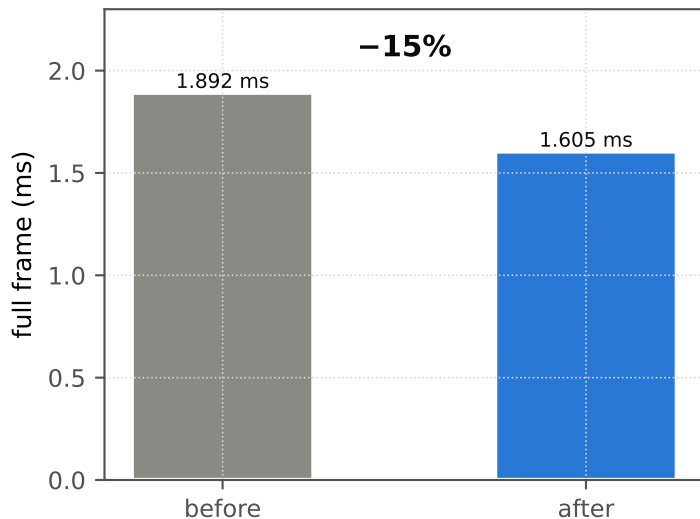


LEFT is the only fully controlled cross-project comparison — same fractal, resolution, iteration count, precision, algorithm class and thread count — so it isolates kernel quality: ruster rejects cardioid/period-2/period-3 interiors and detects cycles; the C++ kernel does neither. RIGHT normalises each project against its own single thread, so it compares scaling shape only, not speed. $\triangle$ The two projects' numbers come from different sessions; quote the left panel as 17–20 $\times$ until all three are re-run together (see other-projects/tri-compare).

fractal_kernel_f32 ablation



Effect on the whole frame



mandelbrot_f32 called the fp64 in_period3_bulb once per pixel. On Ampere GeForce, which runs fp64 at 1/64 the fp32 rate, those 8 operations cost as much as the entire 1000-iteration f32 loop. Replacing them with an f32 predicate is bit-identical — zero differing pixels of 2,073,600 across seven viewports including three centred on the period-3 boundary arc. The full-frame gain is only 15% because the frame is readback-bound (Figure 3).